

SRE CASE STUDY

AZURE ARC HYBRID CLOUD LAB

Parts 1-4: Kubernetes, Azure Arc, Terraform, CI/CD,
Ingress TLS, HPA, NetworkPolicy, RBAC, GitOps, Trivy, Defender

Author: Bartosz Suszko

Company: IT Solutions Bartosz Suszko, Olsztyn, Poland

Date: May 2026 (Parts 1-4)

GitHub: github.com/buth11/sre-homelab-azure-arc

Stack: Kubernetes | Azure Arc | Terraform | KQL | Azure DevOps | FluxCD | Trivy | Defender

Table of Contents

1. Context and Architecture
2. On-Premise Infrastructure — Kubernetes Cluster
3. FinOps — Cloud Cost Control
4. Azure Arc — Hybrid Integration
5. Observability — Container Insights and Azure Monitor
6. KQL Queries — Diagnostics and SLO
7. Load Balancing — Round-Robin Verification
8. Prometheus and Grafana
9. Infrastructure as Code — Terraform
10. Policy as Code — Azure Policy
11. GitHub — Documentation
12. Part 2: Service Principal, Remote State, CI/CD
13. Part 3: Ingress TLS, HPA, NetworkPolicy, RBAC
14. Part 4: GitOps (FluxCD), Trivy, Defender for Cloud
15. Postmortem — POST-001 etcd Split-Brain
16. Results and Conclusions
17. Troubleshooting Log (11 issues)

Chapter 1: Context and Architecture

This project demonstrates building a production-grade hybrid environment on GMKtec EVO-X2 (128 GB RAM) connecting Kubernetes with Microsoft Azure through Azure Arc. Four parts of hands-on practice with documentation of every step and 11 documented incidents.

Layer	Technology	Purpose
Kubernetes	K3s v1.35.4+k3s1	3-node cluster
Ingress	Traefik + TLS	HTTPS domain routing
Autoscaling	HPA (autoscaling/v2)	CPU-based scaling 2-8 replicas
Network Security	NetworkPolicy	Intra-cluster firewall
Access Control	RBAC + ServiceAccount	Least privilege
GitOps	FluxCD 2.8.8	Git-driven reconciliation
Security Scanning	Trivy 0.70.0	Container CVE scanning
Runtime Security	Defender for Cloud	Microsoft threat detection
Cloud Bridge	Azure Arc	Hybrid control plane
Monitoring	Azure Monitor + Prometheus	Observability
IaC	Terraform + Remote State	Infrastructure as Code
CI/CD	Azure DevOps	Pipeline with approval gate

Chapter 2: On-Premise Infrastructure

Node	IP	CPU	RAM
k3s-master (Control Plane)	192.168.122.10	4 vCPU	8 GB
k3s-worker1	192.168.122.11	4 vCPU	16 GB
k3s-worker2	192.168.122.12	4 vCPU	16 GB

Chapter 3: FinOps

Azure Cost Management budget configured BEFORE any resource provisioned — FinOps first principle. Budget: 5 USD/month with email alerts at 50% and 100%. Defined in Terraform — versioned like any other code. Result after full project: 0.00 USD — everything within Azure free tiers.

Chapter 4: Azure Arc

Azure Arc agent installed on the K3s cluster with a single command (`az connectedk8s connect`). The cluster appeared in the Azure portal — K3s-Homelab Connected, v1.35.4+k3s1, 3 nodes, 12 cores. The agent establishes an outbound HTTPS/443 connection with mTLS — no inbound firewall ports needed. After VM restart the cluster reconnected automatically within 3 minutes.

Chapter 5: Observability

Container Insights (AMA — Azure Monitor Agent) installed as an Arc extension — 4 DaemonSet pods on the cluster. Verified: AMA pods Running, 0 restarts. Azure Monitor dashboard shows: CPU avg 1.58%, RAM avg 6.27%, 3 nodes Ready, 29 pods running.

Chapter 6: KQL Queries

5 production-ready KQL queries written and verified:

- KQL-1: Node inventory — 3 nodes Ready, v1.35.4+k3s1
- KQL-2: Pod status per namespace — azure-arc 12 Running, default 3 Running, kube-system 14+2
- KQL-3: Events last 24h — 13 events: Rebooted, Unhealthy (temporary), FailedScheduling
- KQL-4: Pods with restarts — 27 pods after VM reboot, all Running
- KQL-5: SLO calculation — 100% availability (184/184 samples Ready)

Chapter 7: Load Balancing

Round-robin verification: 20 requests across 3 pods — all 3 pods served traffic. 900 parallel connections with zero errors. K3s Klipper Load Balancer distributes traffic evenly across all worker nodes.

Chapter 8: Prometheus & Grafana

kube-prometheus-stack installed via Helm in monitoring namespace. Stack: Prometheus, Grafana, Alertmanager, kube-state-metrics, node-exporter. Grafana dashboard: CPU 2.20%, RAM 12.9%, CPU Quota per namespace. Alert rules configured: High CPU (>80%), High Memory (>85%), Pod down >5 minutes. Alert test: killing a Pod triggered alert within 2 minutes via Alertmanager.

Chapter 9: Terraform IaC

5 Azure resources defined in HCL code: Resource Group, Log Analytics Workspace, cost budget, Action Group (email alerts), OOMKill alert rule. Remote state in Azure Blob Storage (srettfstate5496) with locking and versioning. Terraform import used for brownfield deployment. Terraform Plan: 6 resources, 0 changes after stabilization. Terraform Apply: 3 added, 1 changed.

Chapter 10: Azure Policy

2 policies assigned via Azure CLI (not portal — production practice):

- Policy 1: No root containers — no application can run as root
- Policy 2: Require CPU and memory limits — CPU max 500m, RAM max 512Mi

Mode: DoNotEnforce (Audit) — standard approach on existing environments. Result: both policies Compliant — 100% resource compliance.

Chapter 11: GitHub

Public repository: github.com/buth11/sre-homelab-azure-arc Languages: Shell 58.4%, HCL 41.6%. SSH key authentication — no tokens. All infrastructure changes tracked via Git commits with author and timestamp.

Chapter 12: Part 2 — Service Principal, Remote State, CI/CD

Remote State — sre-homelab.tfstate in Azure Blob Storage:

File: sre-homelab.tfstate — 14.53 KiB, server encrypted, versioning active. State lock visible in logs: 'Acquiring state lock' / 'Releasing state lock' on every terraform plan.

Azure DevOps CI/CD Pipeline:

Variable Group terraform-credentials: ARM_CLIENT_ID, ARM_CLIENT_SECRET (hidden), ARM_SUBSCRIPTION_ID, ARM_TENANT_ID. Environment production: Approval Gate active — all approvers must approve (30d timeout).

Pipeline complete — both stages green, 4m 24s total. Stage 1 (37s): validate + plan + publish artifact. Stage 2 (49s): terraform apply after approval gate.

Chapter 13: Part 3 — Advanced Kubernetes Operations

Ingress with TLS — HTTPS routing by domain

Traefik Ingress Controller configured for HTTPS. Self-signed TLS certificate with SAN generated via openssl, stored as Kubernetes Secret (kubernetes.io/tls). Routing: demo.sre-lab.local → aras-demo-app:8080.

Verification: X-Forwarded-Proto: https, X-Forwarded-Server: traefik-9bcd9bd9-z2mnj, X-Forwarded-Port: 443 visible in browser.

HPA — CPU-based pod autoscaling

HPA configured: minReplicas: 2, maxReplicas: 8, targetCPU: 50%. Load test: ab -n 50000 -c 50 — 1621 req/s over 30 seconds.

Scale-up: CPU 860%/50% → 8 replicas (max). Scale-down: CPU 6%/50% after 5 minutes → 2 replicas (min). Cooldown period works as expected.

NetworkPolicy — intra-cluster firewall

Default deny-all policy for aras-demo-app. Allowed: kube-system (Traefik), monitoring (Prometheus), aras-demo-app replicas. Blocked: everything else.

Verification: unauthorized pod → timeout/blocked ('ZABLOKOWANY — NetworkPolicy działa'). Traefik → application response.

RBAC — least privilege for applications

ServiceAccount: aras-demo-sa. Role: get/list/watch on Pods and ConfigMaps — nothing more. RoleBinding connects SA to Role.

kubectl auth can-i list pods --as=system:serviceaccount:default:aras-demo-sa → yes
kubectl auth can-i delete pods --as=system:serviceaccount:default:aras-demo-sa → no
Least privilege principle confirmed.

Chapter 14: Part 4 — GitOps, Trivy, Defender for Cloud

FluxCD — Git-driven cluster reconciliation

FluxCD 2.8.8 bootstrapped from GitHub repository via SSH key. Four controllers installed in flux-system namespace: helm-controller, kustomize-controller, notification-controller, source-controller — all healthy.

GitOps demo application deployed to gitops-demo namespace. Flux Kustomization watches k8s/part4/apps/ and reconciles every 1 minute.

GitOps test: changed replicas from 2 to 3 in deployment.yaml, committed and pushed. Flux automatically applied the change — third pod appeared without any kubectl apply. Git is the single source of truth.

Trivy — container image CVE scanning

Trivy 0.70.0 installed. Image scanned: traefik/whoami:latest (stdlib Go v1.24.1).

Severity	Count	Key CVE
CRITICAL	1	CVE-2025-68121 — crypto/tls: incorrect certificate validation
HIGH	12	CVE-2025-22874, CVE-2025-61726, CVE-2026-25679 and others
MEDIUM	25	Various net/http, os, crypto/x509 issues
LOW	2	net/http, os module issues

Full scan results stored in k8s/part4/trivy/ (JSON + table format). Root cause: outdated Go stdlib v1.24.1 — fixed in v1.24.13+.

Defender for Cloud — runtime security

Defender for Containers enabled at Standard tier (30-day free trial). Extension microsoft.azuredefender.kubernetes installed on Arc cluster — provisioningState: Succeeded.

7 Defender pods running in mdc namespace: • microsoft-defender-collectors-ds-* (3x — one per node) • microsoft-defender-pod-collector-misc (1x) • microsoft-defender-publisher-ds-* (3x — one per node) All Running, 0 restarts.

Chapter 15: Postmortem POST-001

Field	Value
Severity	P2
Duration	Under 7 minutes
Root Cause	Typo: ks3-worker1 instead of k3s-worker1
Status	Resolved — regex validation in scripts

Chapter 16: Results and Conclusions

Metric	Part 1	Part 2	Part 3	Part 4
Nodes	3/3 Ready	—	—	—
SLO	100%	—	—	—
Cost	\$0.00	\$0.00	\$0.00	\$0.00
Terraform state	Local	Remote (Blob)	—	—
CI/CD	—	Azure DevOps 4m24s	—	—
Ingress TLS	—	—	HTTPS verified	—
HPA	—	—	3→8→2 replicas	—
NetworkPolicy	—	—	Blocked	—
RBAC	—	—	Least privilege	—
GitOps	—	—	—	FluxCD deployed
Image scanning	—	—	—	1 CRITICAL, 12 HIGH
Runtime security	—	—	—	Defender (3 nodes)
Issues	6	8	10	11

Chapter 17: Troubleshooting Log

ISSUE-001 — KQL: CPUCapacityNanoCores column missing

Symptom: project operator: Failed to resolve scalar expression named CPUCapacityNanoCores

Resolution: Use KubeNodeInventory | getschema to verify available columns. Simplified query to existing columns.

ISSUE-002 — Azure Policy: MissingPolicyParameter cpuLimit/memoryLimit

Symptom: (MissingPolicyParameter) policy assignment k8s-resource-limits is missing cpuLimit, memoryLimit

Resolution: Added --params with cpuLimit: 500m and memoryLimit: 512Mi to az policy assignment create.

ISSUE-003 — Terraform: Resource Group already exists (brownfield)

Symptom: Error: A resource with the ID ../SRE-Lab-RG already exists

Resolution: terraform import azurerm_resource_group.sre_lab /subscriptions/.../SRE-Lab-RG

ISSUE-004 — Terraform: Saved plan is stale after import

Symptom: Error: Saved plan is stale — state was changed after plan was created

Resolution: Generate new terraform plan -out=tfplan after any import operation.

ISSUE-005 — Container Insights missing after VM restart

Symptom: AMA pods disappeared from kube-system namespace after full VM reboot

Resolution: az k8s-extension create --name azuremonitor-containers — agents returned in 37s.

ISSUE-006 — POST-001: etcd Split-Brain (ks3 vs k3s)

Symptom: FATA[0034] Node password rejected, duplicate hostname

Resolution: kubectl delete node + rm /etc/rancher/node/ + restart k3s-agent. Scripts updated with regex validation.

ISSUE-007 — Azure DevOps: TerraformInstaller task missing

Symptom: A task is missing: TerraformInstaller

Resolution: Install from Marketplace: ms-devlabs.custom-terraform-tasks.

ISSUE-008 — Pipeline: permission for Variable Group

Symptom: This pipeline needs permission to access a resource

Resolution: Click View → Permit — one-time authorization per pipeline.

ISSUE-009 — sudo does not expand ~ in paths

Symptom: error: Cannot read file ~/ingress/server.crt: no such file or directory

Resolution: Root cause: sudo runs as root which has different home (/root/). Use full path /home/buth11/ instead of ~.

ISSUE-010 — HPA shows cpu: unknown — missing resource requests

Symptom: NAME TARGETS MINPODS MAXPODS REPLICAS — cpu: /50%

Resolution: Added resources.requests to Deployment: cpu: 10m, memory: 32Mi. After rolling update HPA showed correct values.

ISSUE-011 — RBAC RoleBinding: apiRef instead of apiGroup

Symptom: Kubernetes rejected RoleBinding object as invalid — field apiRef does not exist

Resolution: Correct field name is apiGroup: rbac.authorization.k8s.io in roleRef. Fixed in separate YAML file.